

ボールマシン (Ball Machine)

Madina は IOI の参加者を楽しませるために、ボールマシンを発明した。ボールマシンの内部構造は以下の通りである：

- このマシンは N 個のノードで構成され、 0 から $N - 1$ までの番号が付けられている。ノード $N - 1$ を根と呼ぶ。
- $0 \leq u < N - 1$ を満たす各ノード u について、ノード u の親は $P[u]$ であり、 $P[u] > u$ である。ノード u は $P[u]$ の子である。根は親を持たない。子を持たないノードは葉と呼ばれる。

あなたは N の値やノード u の親ノード $P[u]$ についての情報を知らない。代わりに、Madina は葉の数 M と、葉ノードの番号が $0, 1, \dots, M - 1$ であることをあなたに教えてくれた。あなたの課題は、ボールマシンを操作して内部構造を解明することである。

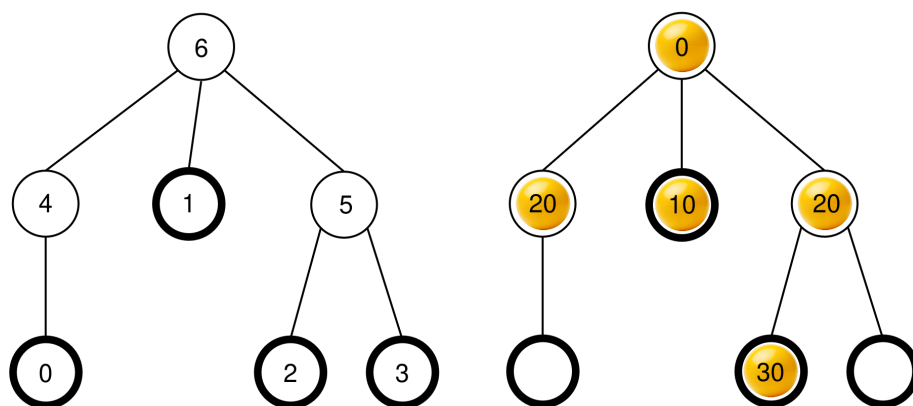
機械の各ノードは最大で 1 つのボールを保持でき、各ボールは非負整数で表される価値を持っている。ノードが価値 x のボールを保持しているとき、そのノードの価値は x であると言う。ノードは、ボールが入っている場合は占有されていると言い、そうでない場合は空いていると言う。最初、すべてのノードが空いている。

実行できる操作は次の 2 種類である。

- insert(U, X)**：価値 X の新しいボールを葉 U に挿入しようとする。
 - 葉 U が占有されている場合、ボールを挿入せずに操作は `false` を返す。
 - 葉 U が空いている場合、ボールはノード U に配置される。その後、ボールは現在のノードの親が空いている限り、その親へ移動することを繰り返す。ボールが根に到達した時、あるいは親が占有されているノードに到達した時に、移動は停止する。操作は `true` を返す。
- collect()**：根が空いている場合（つまりマシンが空の場合）、`collect` は空の配列を返す。そうでない場合、以下に説明する再帰的な関数 `traverse` に従って現在マシン内にあるすべてのボールの価値を配列 S に記録する。最初、配列 S が空の状態で、`traverse(N - 1)` が呼び出される。
 - traverse(u)**：ノード u のボールの価値を S の末尾に追加する。 $c[u]$ を u の子のうち占有されているもののリストとする。 $c[u]$ を、それらのノードのボールの価値の昇順にソートする（複数のノードの価値が等しい場合、順序は問わない）。 $c[u]$ に含まれるそれぞれの v について、 $c[u]$ に登場する順に `traverse( $v$ )` を呼び出す。

この操作が完了するとすべてのボールが機械から取り除かれ、`collect` は S を返す。

例えば、ノードの数が $N = 7$ で、親を表す配列が $P = [4, 6, 5, 5, 6, 6]$ のマシンを考える。左側の図は、ノードの番号が示された空のマシンを表している。右側の図は、一連の insert 操作後のボールの配置例を表している（この操作手順は「例」のセクションで説明されている）。



`collect()` が呼び出されると、根（ノード 6）は占有されているため、 S は $S = []$ として初期化され、`traverse(6)` が呼び出される。

traverse(6)：ノード 6 の価値は 0 である。これを S に追加すると $S = [0]$ になる。ノード 6 の子はノード 4, 1, 5 であり、すべて占有されている。それぞれの価値は 20, 10, 20 である。昇順にソートすると $c[6] = [1, 5, 4]$ となる（ノード 4 と 5 の価値が同じであるため、 $c[6] = [1, 4, 5]$ も有効であることに注意せよ）。次に、この関数では $c[6]$ の各ノードに対して、その順序で `traverse` を呼び出す。

- **traverse(1)**：ノード 1 の価値は 10 である。これを S に追加すると、 $S = [0, 10]$ になる。ノード 1 には占有されている子ノードがないため、この呼び出しは終了する。
- **traverse(5)**：ノード 5 の価値は 20 である。これを S に追加すると、 $S = [0, 10, 20]$ になる。ノード 5 の子ノードのうち占有されているものはノード 2 のみであるため、 $c[5] = [2]$ となり、`traverse(2)` を呼び出す。
 - **traverse(2)**：ノード 2 の価値は 30 である。これを S に追加すると、 $S = [0, 10, 20, 30]$ になる。ノード 2 には占有されている子ノードがないため、この呼び出しは終了する。
 - 実行は `traverse(5)` に戻り、 $c[5]$ 内のすべての子が処理されたため、この呼び出しは終了する。
- **traverse(4)**：ノード 4 の価値は 20 である。これを S に追加すると、 $S = [0, 10, 20, 30, 20]$ になる。ノード 4 には占有されている子ノードがないため、この呼び出しは終了する。

すべての呼び出しが完了し、処理すべきノードはなくなった。最後に、すべてのボールがマシンから取り除かれ、`collect` は配列 $S = [0, 10, 20, 30, 20]$ を返す。

あなたの課題は、ノードの数 N を決定し、マシンの構造を表す親ノードの配列 $R = [R[0], R[1], \dots, R[N-2]]$ を報告することである。ただし、葉と根以外のノードのラベル付けは異なっても良い。厳密には、報告された配列 R は、以下のすべての条件を満たすようにマシンの各

ノード u に相異なるラベル $L[u]$ ($0 \leq L[u] < N$) を割り当てることが可能であれば正しいとみなされる。

- $0 \leq u < M$ または $u = N - 1$ を満たすすべての整数 u に対して $L[u] = u$ である。
- $0 \leq u < N - 1$ を満たすすべての整数 u に対して $L[P[u]] = R[L[u]]$ 。

ここで、 $R[u] > u$ である必要はない。また、解答を報告する時点では、マシンが空である必要がある。

K を実行された `collect` 操作の回数とし、 B をすべての `insert` 操作におけるボールの価値の最大値とする。 $C = K + B$ とする。 C は 1000 を超えてはいけない。一部の小課題における得点は C の値によって決まる。

実装の詳細

あなたは以下の関数を実装する必要がある。

```
std::vector<int> find_structure(int M)
```

- M ：マシンの葉の数。
- この関数は、各テストケースに対してちょうど 1 回呼び出される。
- この関数は、正しい親ノードの配列を表す長さ $N - 1$ の配列 $R = [R[0], R[1], \dots, R[N - 2]]$ を返す必要がある。

マシンとやり取りするために以下の 2 種類の関数を呼び出すことができる。

1 つ目の関数は以下である。

```
bool insert(int U, int X)
```

- U ：ボールを挿入する葉の番号。 $0 \leq U < M$ を満たす必要がある。
- X ：ボールの整数の価値。 $0 \leq X \leq 1000$ を満たす必要がある。
- この関数は、ボールが正常に挿入された場合は `true` を返し、葉 U が既に占有されていた場合は `false` を返す。
- この関数は最大で 500 000 回まで呼び出すことができる。

2 つ目の関数は以下である。

```
std::vector<int> collect()
```

- この関数は、挿入されたすべてのボールの価値を記録し、マシンを空にした上で、記録された配列 S を返す。

プログラムの実行中に C の値が 1000 を超えた場合、あなたの解答は `Output isn't correct: Too many resources used` という判定を受ける。

`find_structure` が呼び出される前に、マシンの構造は固定されている．また、採点プログラムは、**決定論的**である．すなわち、2 回実行し両方の実行で同じ操作の列が実行された場合、`collect` は同じ配列を返す．

制約

- $2 \leq N \leq 1000$
- $1 \leq M \leq 200$
- $M < N$

小課題

小課題	得点	追加の制約
1	5	根はちょうど M 個の子を持つ．
2	10	$M \leq 3$
3	25	$N \leq 200$, $M \leq 45$
4	60	追加の制約はない．

小課題 3,4 では、得点は C の値によって次のように決まる．

小課題 3 (25 点)

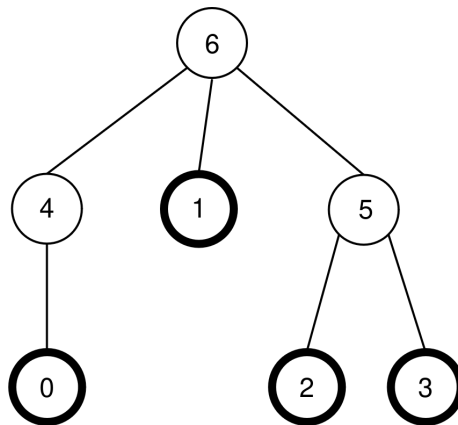
条件	得点
$1000 < C$	0
$45 < C \leq 1000$	13
$C \leq 45$	25

小課題 4 (60 点)

条件	得点
$1000 < C$	0
$200 < C \leq 1000$	7
$71 < C \leq 200$	$47 - \frac{C}{5}$
$44 < C \leq 71$	$104 - C$
$C \leq 44$	60

例

問題文の説明で用いたマシンと同じものを考える．すなわち， $N = 7$ ， $M = 4$ ， $P = [4, 6, 5, 5, 6, 6]$ の場合を考える．このマシンは次の図に再び示されている．



採点プログラムは次の呼び出しを行う．

```
find_structure(4)
```

この関数は、以下の操作の列を実行できる．

1. **insert(0, 0)**：葉ノード 0 は空いているため、価値 0 のボールが葉ノード 0 に配置される．ノード $P[0] = 4$ は空いているため、ボールはノード 4 に移動する．ノード $P[4] = 6$ は空いているため、ボールはノード 6 に移動する．ノード 6 は根であるため、移動は停止する．この関数は `true` を返す．
2. **insert(3, 20)**：葉ノード 3 は空いているため、価値 20 のボールが葉ノード 3 に配置される．ノード $P[3] = 5$ は空いているため、ボールはノード 5 に移動する．ノード $P[5] = 6$ は占有されているため、移動は停止する．この関数は `true` を返す．
3. **insert(1, 10)**：葉ノード 1 は空いているため、価値 10 のボールが葉ノード 1 に配置される．ノード $P[1] = 6$ は占有されているため、ボールはノード 1 に留まる．この関数は `true` を返す．
4. **insert(2, 30)**：葉ノード 2 は空いているため、価値 30 のボールが葉ノード 2 に配置される．ノード $P[2] = 5$ は占有されているため、ボールはノード 2 に留まる．この関数は `true` を返す．
5. **insert(1, 25)**：葉ノード 1 は占有されているため、ボールは葉 1 に配置されない．この関数は `false` を返す．
6. **insert(0, 20)**：葉ノード 0 は空いているため、価値 20 のボールが葉ノード 0 に配置される．ノード $P[0] = 4$ は空いているため、ボールはノード 4 に移動する．ノード $P[4] = 6$ は占有されているため、移動は停止する．この関数は `true` を返す．

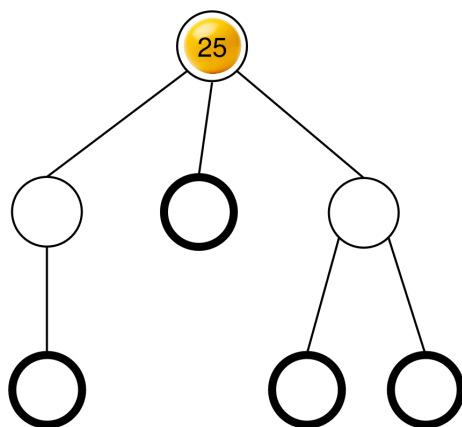
結果として得られたボールの配置はすでに問題文の説明に表示されている．

次に、以下の呼び出しを行うことができる．

```
collect()
```

この呼び出しの実行については、既に問題文の説明にあるように、マシンを空にした上で配列 $S = [0, 10, 20, 30, 20]$ を返す。

次に、`insert(2, 25)` を呼び出すことができる。葉ノード 2 は空いているため、価値 25 のボールが葉ノード 0 に配置される。このボールはノード 5 に移動し、次にノード 6 に移動して停止する。呼び出しは `true` を返す。結果として得られるボールの配置は次の図に示されている。



最後に、この関数は `collect()` を呼び出すことができ、 $S = [25]$ を返してマシンを空にする。

`find_structure` が返すことのできる、正しい親ノードの配列は 2 つある。 $R = P = [4, 6, 5, 5, 6, 6]$ (ラベル $L = [0, 1, 2, 3, 4, 5, 6]$ に対応) と、 $R = [5, 6, 4, 4, 6, 6]$ (ラベル $L = [0, 1, 2, 3, 5, 4, 6]$ に対応) の 2 つである。この例では、 $K = 2$ および $B = 30$ であるため、 $C = 32$ となる。

採点プログラムのサンプル

入力形式：

```
N M
P[0] P[1] P[2] ... P[N-2]
```

出力形式：

```
K B C
Q
R[0] R[1] R[2] ... R[Q-1]
```

ここで、 Q は `find_structure` が返す配列 R の長さである。